

Evidencia de Cola o Queue

Dialog

?

×

Ingresar

Eliminar

Imprimir primer valor

Imprimir fila

Lista vacía

Imprimir último valor

Dato:

Head -> 5 <- Tail

Dialog

?

×

Ingresar

Eliminar

Imprimir primer valor

Imprimir fila

Lista vacía

Imprimir último valor

Dato:

Head -> 5 -> 8 <- Tail

Dialog

?

×

Ingresar

Eliminar

Imprimir primer valor

Imprimir fila

Lista vacía

Imprimir último valor

Dato:

Head -> 5 -> 8 -> 3 <- Tail

Dialog

?

×

Ingresar

Eliminar

Imprimir primer valor

Imprimir fila

Lista vacía

Imprimir último valor

Dato:

Head -> 5

Dialog

?

×

Ingresar

Eliminar

Imprimir primer valor

Imprimir fila

Lista vacía

Imprimir último valor

Dato:

Tail -> 3

Dialog

?

×

Ingresar

Eliminar

Imprimir primer valor

Imprimir fila

Lista vacía

Imprimir último valor

Dato:

Head -> 8 -> 3 <- Tail

Dialog ? X

Ingresar

Eliminar

Imprimir primer valor

Imprimir fila

Lista vacía

Imprimir último valor

Dato:

Head -> 8 -> 3 -> 9 <- Tail

Dialog ? X

Ingresar

Eliminar

Imprimir primer valor

Imprimir fila

Lista vacía

Imprimir último valor

Dato:

Head -> 3 -> 9 <- Tail

Evidencia de Prompt

Puedes revisar el siguiente código de mi clase Queue. Verifica si cumple con las operaciones enqueue,

dequeue, printQueue, firstQueue y lastQueue. También revisa si respeta programación orientada a

objetos y si no afecta la clase Stack existente.

Necesito que me indiques:

1. Errores encontrados.
2. Mejoras mínimas recomendadas.
3. Si el comportamiento FIFO está correctamente implementado.
4. Casos de prueba que debo ejecutar.

Código a revisar:

```
from estructuras.lineales.nodo import Node

class Queue(object):

    def __init__(self):
        self.head = None
        self.tail = None

    def enqueue(self, data):
        new_node = Node(data) #Se crea nodo con el dato otorgado
        if self.head is None and self.tail is None: #Si ambos están vacíos, ambos apuntarán al nodo
            self.head = new_node
            self.tail = new_node
        else:
            self.tail.next = new_node #Si no, el next de la cola apuntará al nodo y la cola cola hará lo mismo
            self.tail = new_node

        return "Se ha agregado el nodo a la fila"

    def dequeue(self):
```

```
    if self.head is None and self.tail is None: #En caso de no haber nada en fila, solo mostrará el mensaje
```

```
        return "No existen valores en la fila"
```

```
    else:
```

```
        self.head = self.head.next #Head apuntará al nodo siguiente eliminando el anterior
```

```
        return "Se ha eliminado el nodo de la fila"
```

```
def firstqueue(self):
```

```
    if self.head is None and self.tail is None:
```

```
        return "No existen valores en la fila"
```

```
    else:
```

```
        return f"Head -> {self.head.data}" #Imprime la cabeza
```

```
def lastqueue(self):
```

```
    if self.head is None and self.tail is None:
```

```
        return "No existen valores en la fila"
```

```
    else:
```

```
        return f"Tail -> {self.tail.data}" #Imprime la cola
```

```
def printqueue(self):
```

```
    if self.head is None and self.tail is None:
```

```
        return "No existen valores en la fila"
```

```
    else:
```

```
        temp = self.head #Se crea un temporal
```

```
        x = "Head -> " #Se crea un texto
```

```
        while temp != None: #Se hace un bucle hasta que temporal sea un None
```

```
            if temp.next is None: #Si el siguiente de temporal es None se agrega el dato y termina el texto sin coma
```

```
                x += f"{temp.data} <- Tail"
```

```
                temp = temp.next #Avanza el temporal apuntando a su next
```

```
            else:
```

```
                x += f"{temp.data} -> " #En caso de no ser None se agrega el dato con coma
```

```
                temp = temp.next #El temp apuntará a su next hasta que su valor sea None
```

```
        return x #Imprime todo lo que encontró
```

```
def isEmpty(self):  
    if self.head is None and self.tail is None: #Si está vacía la lista  
        return True #Imprime True  
    else:  
        return None #Imprime None
```

Evidencia de Respuesta

En general, tu implementación está **bien encaminada** y sí representa una cola enlazada usando programación orientada a objetos. No veo nada que pueda afectar una clase Stack existente, siempre que ambas usen la misma clase Node y cada una mantenga sus propios atributos (head, tail o top).

A continuación te hago una revisión punto por punto.

1. Errores encontrados

Error 1. dequeue() no actualiza tail cuando se elimina el último nodo

Este es el error más importante.

Supongamos que la cola tiene un solo elemento.

La solución mínima sería:

```
self.head = self.head.next
```

```
if self.head is None:
```

```
    self.tail = None
```

2. ¿Está correctamente implementado el comportamiento FIFO?

Sí.

La lógica FIFO está bien.

enqueue()

Agrega siempre al final.

Head

10 -> 20 -> 30

↑

Tail

Al hacer

enqueue(40)

queda

10 -> 20 -> 30 -> 40

↑

Tail

Correcto.

dequeue()

Elimina siempre el primero.

Head

10 -> 20 -> 30

Después

20 -> 30

Correcto.

Mientras corrigas el problema del último nodo, la política FIFO queda completamente correcta.

3. Casos de prueba recomendados

Caso 1

Cola vacía

q = Queue()

q.isEmpty()

q.printqueue()

q.dequeue()

q.firstqueue()

q.lastqueue()

Debe indicar que está vacía.

Caso 2

Insertar un elemento

q.enqueue(10)

Esperado

Head -> 10 <- Tail

Caso 3

Insertar varios

q.enqueue(10)

q.enqueue(20)

q.enqueue(30)

Esperado

Head -> 10 -> 20 -> 30 <- Tail

Caso 4

Eliminar uno

10

20

30

Después de

q.dequeue()

Debe quedar

Head -> 20 -> 30 <- Tail

Caso 5

Eliminar todos

10

20

30

hacer

dequeue()

dequeue()

dequeue()

Debe quedar

Head = None

Tail = None

Este caso sirve para verificar el error mencionado.

Caso 6

Volver a insertar después de vaciar

enqueue(10)

dequeue()

enqueue(50)

Debe imprimirse

Head -> 50 <- Tail

Si no corregiste tail, aquí aparecerán errores.

Caso 7

FIFO

enqueue(1)

enqueue(2)

enqueue(3)

dequeue()

dequeue()

Debe permanecer únicamente

3

Eso demuestra que salió primero el 1 y luego el 2.

Explicación

Los señalamientos y código generado por la Inteligencia Artificial fueron tomado únicamente en un solo punto, en el que se señalaba el posible bug de eliminar un valor al encontrarse un solo valor:

```
self.head = self.head.next
```

```
if self.head is None:
```

```
    self.tail = None
```

El código fue implementado directamente únicamente con esa modificación y ajustes de nombres de variables.

El proceso es simple, se lee la entrada que se genere en la caja de texto de Qt y se agrega a la lista entrando como cabeza o cola cuando no hay ningún valor o entra como cola cuando ya existen valores al presionar el botón de Ingresar

Al presionar el botón de salida entra el código donde la cabeza apunta al next de este mismo para eliminar el valor que se encuentra actualmente en la cabeza y que la cabeza apunte a siguiente valor

Al presionar Imprimir primer elemento únicamente se imprime el valor o dato que se encuentra en la cabeza. De igual forma, al presionar el botón de Imprimir último elemento únicamente se imprime el valor o dato que se encuentra en la cola.

Al presionar Lista vacía el código busca si en la lista head y tail tienen un valor None, en caso de ser así la lista está vacía y lanza un True, en caso contrario de tener un valor cualquiera, lanzará un False.

Finalmente el botón de Imprimir Fila activa un temporal que avanza en toda la lista como temp.next con la condición de ser distinto de None su valor, agregando el dato de cada punto a una variable, por lo que recorrerá toda la lista completa, una vez llegue al último valor entra el valor de None y el temporal se detendrá e imprimirá la variable que guardó todos los valores obtenidos.